

TRAVAUX PRATIQUES IX

Reconnaissance de caractères

Dans ce TP, on applique l'algorithme $k - NN$ à l'une de ses plus fameuses applications : la reconnaissance de caractères. Plus précisément, la reconnaissance de chiffres. Nous utiliserons la base de données MNIST (acronyme de *Mixed National Institute of Standards and Technology*).

La base de donnée MNSI est composée d'images de chiffres. Toutes les images ont la même dimension, et contiennent un seul chiffre. Voici un extrait de plusieurs chiffres de cette base :



FIGURE IX.1 – Des observations issues de MNIST. Vous devinerez aisément les étiquettes, k -NN aussi (?).

Vous trouverez un miroir de téléchargement pour cette base de données ici : <https://github.com/cvdfoundation/mnist>¹. Vous devez télécharger les **quatre** parties, puis les décompresser avec la commande terminale `gzip -dk file.gz`. Protip : `gzip -dk *.gz` les décompresse toutes d'un coup.

1. Plusieurs fichiers sont fournis avec ce TP : `libtoolbox.so`, `toolboc.h`, `administratif.c`, `administratif.h`, `main.c`, `Makefile` vous sont fournis avec ce TP. Mettez-les dans un même dossier, avec la base MNIST (comme indiqué ci-dessus). C'est là que l'on travaillera.

Décrivons rapidement la base MNIST :

- `train-images-idx3-ubyte` est un fichier qui contient les 50k observations du jeu d'entraînement (ce sont des images de chiffres).
- `train-labels-idx1-ubyte` contient les 50k étiquettes de l'entraînement (les bons chiffres donc).
- `t10k-images-idx3-ubyte` contient les 10k observations du jeu de test.
- `t10k-labels-idx1-ubyte` contient les 10k étiquettes du test.

Vous remarquez donc que MNIST est déjà découpé en entraînement/test. En fait, cette base de données a acquis une telle popularité qu'elle est devenue un standard pour mesurer la performance d'algorithmes. Imposer la séparation entraînement/test permet donc que tout le monde s'entraîne et se teste sur la même partie de la base de données, et donc de rendre possible les comparaisons de performances.

1. On pourrait aussi la télécharger directement depuis le site de Yann Lecun avec `wget` en connaissant son nom, c'est sa source originelle ; mais vous perdriez trop de temps.

A Rappels et utilitaires

A.1 Fourni avec ce TP

Pour ce TP, beaucoup de code est pré-codé. Faisons un rapide overview :

- `toolbox.so` contient des fonctions permettant d'afficher une image, et d'ouvrir un fichier de MNIST.
Vous n'avez pas besoin de vous intéresser à ce fichier.
- `administratif.h` (et son implémentation `administratif.c`) contient les fonctions qui font les lectures dans les fichiers de MNIST (ouverts grâce à `toolbox`), et affichent le résultat de kNN.
Cependant, ce fichier ne contient *pas* d'implémentation de kNN : ce sera votre travail ! Vous travaillerez dans `main.c`, et les fonctions que vous coderez seront passées en argument aux fonctions de `administratif.c` (qui à l'aide de votre kNN classifient des tests et afficheront le résultat).
Dans la variante normale du TP, vous n'avez pas à vous intéresser à ce fichier.
- `main.c` : c'est le fichier qui contient les fonctions à trous que vous devrez coder, ainsi que le `main` (mais ce `main` ne fait pas grand chose à part appeler des fonctions de `administratif`). Vous pouvez toucher au `main` si vous le souhaitez.
C'est le fichier intéressant !
- `Makefile` : lancez `make` pour compiler puis, si la compilation a réussi, `make run` pour exécuter.
Vous voudrez sans doute modifier la règle `run` du `Makefile`.

Remarque : Variante étoilé du TP : dans la version étoilée du TP, une des fonctions de `administratif.c` n'est pas implémentée : le mode `matrix`. Vous devrez donc la programmer.

A.2 Format de données

Les images que vous manipulez sont des images en noir et blanc : chacun de leur pixel est défini par son niveau de gris, qui est un entier non-signé 8bit (de type `uint8_t`).

De plus, bien que les images aient l'air en 2D à nos yeux humains, elles sont stockées dans les fichiers sous une forme linéarisée. Il s'agit de stocker la matrice des pixels comme une seule grande ligne, composée des lignes de la matrice les unes après les autres :

$$\begin{array}{ccc}
 \begin{pmatrix} (0,0) & (0,1) & (0,2) \\ (1,0) & (1,1) & (1,2) \\ (2,0) & (2,1) & (2,2) \end{pmatrix} & & ((0,0) \ (0,1) \ (0,2) \ (1,0) \ (1,1) \ (1,2) \ (2,0) \ (2,1) \ (2,2)) \\
 \text{(a) Une matrice (en 2D)} & & \text{(b) Forme linéarisée de la matrice}
 \end{array}$$

FIGURE IX.2 – Linéarisation d'une matrice

En conséquence :

- Dans ce TP, **chaque observation** (c'est à dire chaque image) **est un tableau de niveau de gris**, donc de type `uint8_t*`.
Il est garanti que toutes les observations, qu'elles soient dans le jeu d'entraînement ou le jeu de test, contiennent autant de pixel.
- En conséquence, **un tableau d'observation est un tableau de tableaux de niveaux de gris**, donc de type `uint8_t**`.

Précisons également que :

- Les classes (les étiquettes) associées aux observations sont les entiers 0, 1, ..., 9. Autrement dit, **une classe est de type `uint8_t` et un tableau de classes est de type `uint8_t`**.

Ces 3 points sont importants à comprendre, car pour coder kNN vous aurez besoin de manipuler des observations et des étiquettes.

A.3 Lecture de la ligne de commande

Cette section explique un morceau de code pré-codé.

Rappelons que la fonction `main` prend deux arguments : `argc` et `argv`, qui servent à lire la ligne de commande tapée dans le terminal pour lancer le programme. `argc` est le nombre de « mots » de cette ligne (les mots sont définis comme étant séparés par des espaces), et `argv` est un tableau contenant chacun de ces mots.

Ainsi, si on lance l'exécutable en tapant :

Terminal

```
1 ./exe display file 3 10
```

Alors `argc` vaudra 5 et `argv` est le tableau contenant `"./exe"` puis `"display"` puis `"file"` puis `"3"` puis `"10"`. (En particulier, `argv` est un tableau qui a `argc` cases.)

Précisons également que la fonction `atoi` permet de convertir une chaîne de caractères contenant un entier (par exemple `"10"`) en cet entier (`10`).

B Codons kNN

B.1 Prise en main

1. Lisez le commentaire de documentation de la fonction `main` (écrit juste avant `int main(...)`). Modifiez le `Makefile` pour que sa règle `run` demande d'afficher l'image numéro 3 du jeu d'entraînement. Puis compilez et exécutez. Vous pouvez aussi choisir de lancer directement la bonne commande en terminal plutôt que de modifier le `Makefile`. Pour le zoom, vous pouvez utiliser une valeur initiale de 10 (libre à vous de manipuler ceci à votre convenance).

- a. Pour comparer des images, on utilisera le carré de la distance euclidienne. Selon vous, pourquoi le carré ?
- b. Quels sont les entiers représentables dans le type `uint8_t` ?
- c. La fonction `distance` doit prendre en arguments deux vecteurs et calculer le carré de la distance euclidienne entre eux. Implémentez-la.

En pratique, ces vecteurs seront les images (on veut trouver les plus proches images) et sont donc de type `uint8_t`.

⚠ Vous ferez attention au fait que la somme/différence/carré de deux niveaux de gris peut tout à fait déborder du type `uint8_t`. Il est donc conseillé de convertir ces quantités en entiers signés 32 bits, à savoir des `int`. Pour rappel, si `x` est un élément que l'on veut convertir dans un type `new_type`, on peut utiliser la syntaxe suivante :

```
1 new_type x_converti = (new_type) x;
```

3. a. Complétez la fonction `nearest_obs`. Elle prend en entrée :
 - Un tableau d'observations (c'est à dire d'images codées comme des tableaux de niveaux de gris) nommé `observations_train`. Il contient `n_train` observations. Il s'agit des observations d'entraînement.
 - `test` une image. Il s'agit d'une observation du jeu de test.
 - `n_train` précédemment expliqué.
 - `taille` le nombre de pixels de chaque image.
 - `d_ret` un pointeur vers un entier. Il sert à « renvoyer » une deuxième valeur en modifiant son contenu.

La fonction doit en sortie :

 - Renvoyer l'indice de `observations_train` qui correspond à l'observation la plus proche de `test`.
 - Si `d_ret != NULL`, stocker dans `*d_ret` la distance associée.
- b. Modifiez la règle `run` du `Makefile` pour qu'elle lance le mode 1NN, par exemple sur la 3ème image. Compilez et testez !

B.2 Optimisation

Parlons un peu d'optimisation compilateur. Un compilateur est un programme qui traduit un code source en un code cible. Il peut essayer d'optimiser le code cible.

Par défaut, gcc n'optimise pas. C'est le niveau d'optimisation 0. Il possède trois niveau d'optimisation (de plus en plus poussée) : 1, 2 et 3. L'option de gcc nommée -O0 met l'optimisation à 0 (par défaut), -O1 à 1, etc.

Vous pouvez modifier les options passées à gcc en modifiant la variable CFLAGS du Makefile. Une fois que vous avez un code qui semble correct, n'hésitez pas à dégainer -O3 pour l'accélérer !

(HP) Les optimisations peuvent parfois faire *trop* de zèle, et des bugs peuvent apparaître avec l'option -O3 alors qu'ils étaient absents à des niveaux moindres. Il existe des moyens de dire au compilateur de ne pas optimiser certains morceaux du code (dont le mot clef volatile).

Nous ne devrions pas rencontrer de telles difficultés aujourd'hui.

B.3 k c'est mieux que 1

4. Complétez la fonction `k_nearest_obs`. Elle prend en entrée :

- `observations_train`, `test`, `n_train`, `taille` comme précédemment.
- `k` le méta-paramètre de l'algorithme kNN.
- `indices_voisins` et `distances` deux tableaux de longueur `k`. Vous pouvez (et devrez) réécrire leur contenu.

Et doit en sortie :

- Ne rien renvoyer.
- Stocker dans `indices_voisins` les indices des `k` plus proches voisins dans `observations_train` de `test`.
- Stocker dans `distances` les distances associées.

5. Complétez `classify_kNN`. Elle prend en entrée :

- Le tableau des observations du jeu d'entraînement `observations_train` ainsi que le tableaux des étiquettes associées `etiquettes_train`. Ils sont tous deux de longueur `n_train`.
- `test` une image du jeu de test à classifier.
- `taille` le nombre de pixels de chaque image.
- `k` le méta-paramètre de kNN.
- `indices_voisins` et `distances` deux tableaux de longueur `k` quelconques (contenus arbitraires). Vous devrez commencer par appeler la fonction précédente pour remplir ces deux tableaux correctement.

La fonction doit renvoyer en sortie la classe (étiquette) majoritaire parmi les `k` plus proches images d'entraînement de l'image de test.

6. Modifiez la règle run du Makefile pour qu'elle lance le mode kNN, compilez, testez.
7. (Version étoilée uniquement :) ouvrez `administratif.h` et `administratif.c`. La fonction `mode_matrix` n'est pas implémentée, vous devez la coder. Vous pouvez vous inspirer de `mode_kNN`.
8. (Tout le monde :) Modifiez la règle run pour qu'elle lance le mode matrice, compilez, testez. Essayez de trouver le meilleur `k` =)